JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class BufferedInputStream

java.lang.Object
    java.io.InputStream
        java.io.FilterInputStream
            java.io.BufferedInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

```
public class BufferedInputStream
extends FilterInputStream
```

A `BufferedInputStream` adds functionality to another input stream-namely, the ability to buffer the input and to support the `mark` and `reset` methods. When the `BufferedInputStream` is created, an internal buffer array is created. As bytes from the stream are read or skipped, the internal buffer is refilled as necessary from the contained input stream, many bytes at a time. The `mark` operation remembers a point in the input stream and the `reset` operation causes all the bytes read since the most recent `mark` operation to be reread before new bytes are taken from the contained input stream.

API Note:

Once wrapped in a `BufferedInputStream`, the underlying `InputStream` should not be used directly nor wrapped with another stream.

Since:

1.0

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	**`buf`**	The internal buffer array where the data is stored.
`protected int`	**`count`**	The index one greater than the index of the last valid byte in the buffer.
`protected int`	**`marklimit`**	The maximum read ahead allowed after a call to the `mark` method before subsequent calls to the `reset` method fail.

protected int	markpos	The value of the pos field at the time the last mark method was called.
protected int	pos	The current position in the buffer.

Fields declared in class FilterInputStream

in

Constructor Summary

Constructors

Constructor	Description
BufferedInputStream (InputStream in)	Creates a BufferedInputStream and saves its argument, the input stream in, for later use.
BufferedInputStream (InputStream in, int size)	Creates a BufferedInputStream with the specified buffer size, and saves its argument, the input stream in, for later use.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	available()	Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
void	close()	Closes this input stream and releases any system resources associated with the stream.

void	**mark**(int readlimit)	See the general contract of the mark method of InputStream.
boolean	**markSupported**()	Tests if this input stream supports the mark and reset methods.
int	**read**()	See the general contract of the read method of InputStream.
int	**read**(byte[] b, int off, int len)	Reads bytes from this byte-input stream into the specified byte array, starting at the given offset.
void	**reset**()	See the general contract of the reset method of InputStream.
long	**skip**(long n)	See the general contract of the skip method of InputStream.

Methods declared in class FilterInputStream

read

Methods declared in class InputStream

nullInputStream, readAllBytes, readNBytes, readNBytes, skipNBytes, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

buf

protected volatile byte[] buf

The internal buffer array where the data is stored. When necessary, it may be replaced by another array of a different size.

count

```
protected int count
```

The index one greater than the index of the last valid byte in the buffer. This value is always in the range `0` through `buf.length`; elements `buf[0]` through `buf[count-1]` contain buffered input data obtained from the underlying input stream.

pos

```
protected int pos
```

The current position in the buffer. This is the index of the next byte to be read from the `buf` array.

This value is always in the range `0` through `count`. If it is less than `count`, then `buf[pos]` is the next byte to be supplied as input; if it is equal to `count`, then the next `read` or `skip` operation will require more bytes to be read from the contained input stream.

See Also:

`buf`

markpos

```
protected int markpos
```

The value of the `pos` field at the time the last `mark` method was called.

This value is always in the range `-1` through `pos`. If there is no marked position in the input stream, this field is `-1`. If there is a marked position in the input stream, then `buf[markpos]` is the first byte to be supplied as input after a `reset` operation. If `markpos` is not `-1`, then all bytes from positions `buf[markpos]` through `buf[pos-1]` must remain in the buffer array (though they may be moved to another place in the buffer array, with suitable adjustments to the values of `count`, `pos`, and `markpos`); they may not be discarded unless and until the difference between `pos` and `markpos` exceeds `marklimit`.

See Also:

`mark(int)`, `pos`

marklimit

```
protected int marklimit
```

The maximum read ahead allowed after a call to the `mark` method before subsequent calls to the `reset` method fail. Whenever the difference between `pos` and `markpos` exceeds `marklimit`, then the mark may be dropped by setting `markpos` to `-1`.

See Also:

`mark(int)`, `reset()`

Constructor Details

BufferedInputStream

```
public BufferedInputStream(InputStream in)
```

Creates a `BufferedInputStream` and saves its argument, the input stream `in`, for later use. An internal buffer array is created and stored in `buf`.

Parameters:

`in` - the underlying input stream.

BufferedInputStream

```
public BufferedInputStream(InputStream in,
                           int size)
```

Creates a `BufferedInputStream` with the specified buffer size, and saves its argument, the input stream `in`,
for later use. An internal buffer array of length `size` is created and stored in `buf`.

Parameters:

`in` - the underlying input stream.

`size` - the buffer size.

Throws:

`IllegalArgumentException` - if `size <= 0`.

Method Details

read

```
public int read()
         throws IOException
```

See the general contract of the `read` method of `InputStream`.

Overrides:

`read` in class `FilterInputStream`

Returns:

the next byte of data, or `-1` if the end of the stream is reached.

Throws:

`IOException` - if this input stream has been closed by invoking its `close()` method, or an I/O error occurs.

See Also:

`FilterInputStream.in`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads bytes from this byte-input stream into the specified byte array, starting at the given offset.

This method implements the general contract of the corresponding `read` method of the `InputStream` class. As an additional convenience, it attempts to read as many bytes as possible by repeatedly invoking the `read` method of the underlying stream. This iterated `read` continues until one of the following conditions becomes true:

- The specified number of bytes have been read,
- The `read` method of the underlying stream returns `-1`, indicating end-of-file, or
- The `available` method of the underlying stream returns zero, indicating that further input requests would block.

If the first `read` on the underlying stream returns `-1` to indicate end-of-file then this method returns `-1`. Otherwise, this method returns the number of bytes actually read.

Subclasses of this class are encouraged, but not required, to attempt to read as many bytes as possible in the same fashion.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b` - destination buffer.

`off` - offset at which to start storing bytes.

`len` - maximum number of bytes to read.

Returns:

the number of bytes read, or `-1` if the end of the stream has been reached.

Throws:

`IOException` - if this input stream has been closed by invoking its `close()` method, or an I/O error occurs.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

See Also:

`FilterInputStream.in`

skip

```
public long skip(long n)
          throws IOException
```

See the general contract of the `skip` method of `InputStream`.

Overrides:

`skip` in class `FilterInputStream`

Parameters:

`n` - the number of bytes to be skipped.

Returns:

the actual number of bytes skipped.

Throws:

`IOException` - if this input stream has been closed by invoking its `close()` method, `in.skip(n)` throws an IOException, or an I/O error occurs.

See Also:

`InputStream.skipNBytes(long)`

available

```
public int available()
              throws IOException
```

Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream. The next invocation might be the same thread or another thread. A single read or skip of this many bytes will not block, but may read or skip fewer bytes.

This method returns the sum of the number of bytes remaining to be read in the buffer (`count - pos`) and the result of calling the `in.available()`.

Overrides:

`available` in class `FilterInputStream`

Returns:

an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking.

Throws:

`IOException` - if this input stream has been closed by invoking its `close()` method, or an I/O error occurs.

mark

```
public void mark(int readlimit)
```

See the general contract of the `mark` method of `InputStream`.

Overrides:

`mark` in class `FilterInputStream`

Parameters:

`readlimit` - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

`reset()`

reset

```
public void reset()
           throws IOException
```

See the general contract of the `reset` method of `InputStream`.

If `markpos` is `-1` (no mark has been set or the mark has been invalidated), an `IOException` is thrown. Otherwise, `pos` is set equal to `markpos`.

Overrides:

`reset` in class `FilterInputStream`

Throws:

`IOException` - if this stream has not been marked or, if the mark has been invalidated, or the stream has been closed by invoking its `close()` method, or an I/O error occurs.

See Also:

`mark(int)`

markSupported

```
public boolean markSupported()
```

Tests if this input stream supports the `mark` and `reset` methods. The `markSupported` method of `BufferedInputStream` returns `true`.

Overrides:

`markSupported` in class `FilterInputStream`

Returns:

a `boolean` indicating if this stream type supports the `mark` and `reset` methods.

See Also:

`InputStream.mark(int)`, `InputStream.reset()`

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream. Once the stream has been closed, further read(), available(), reset(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `FilterInputStream`

Throws:

`IOException` - if an I/O error occurs.

See Also:

`FilterInputStream.in`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.